Study Material — Part 3: MLflow 3 for GenAI

Evaluation, Tracing, Prompt Registry, Custom Scorers

Deliverable 4 (Part 3 of several) — Databricks GenAI Engineer Associate

This covers Section 6 (Evaluation & Monitoring) objectives heavily, plus Section 3 and 4 objectives touching MLflow. This is the most technically precise section — exact API names and patterns are directly tested.

Topic 11: MLflow 3 GenAI Overview

What is it?

MLflow is the open-source platform Databricks uses for the full ML lifecycle — experiment tracking, model registration, deployment, and evaluation. **MLflow 3** introduced a dedicated GenAI evaluation suite that goes far beyond the older `mlflow.evaluate()` API, with new first-class concepts for agents, scorers, and traces.

Why does the version matter?

The exam tests **MLflow 3 GenAI-specific APIs**. Several questions in your practice set have wrong-answer options that describe valid MLflow 2.x patterns but are incorrect for MLflow 3. Knowing which API belongs to which generation prevents you from picking a plausible-sounding but outdated answer.

Key MLflow 3 GenAI components you must know

1. `mlflow.genai.evaluate()` — the primary evaluation function for GenAI apps
2. `@mlflow.genai.scorer` decorator — define custom scorers as Python functions
3. **Built-in judges** — pre-built LLM-as-judge scorers for common quality dimensions
4. **MLflow Tracing** — automatic step-by-step execution logging for agents and chains
5. **Prompt Registry** — versioned storage and lifecycle management for prompt templates

---

Topic 12: MLflow Tracing

What is it?

MLflow Tracing automatically records every step of an agent's or chain's execution — each LLM call, tool invocation, retrieval query, and intermediate result — as a structured, searchable trace that you can inspect in the MLflow UI.

Why is it used?

When a multi-step agent fails or produces a bad result, you need to know *which step* went wrong. Tracing gives you step-by-step execution visibility without adding manual logging code everywhere in your application.

How does it work?

- For supported frameworks (LangChain, LangGraph, OpenAI SDK, Databricks Agent Framework), MLflow Tracing is enabled with **one line** (often just setting an autolog flag or using a context manager)
- Each "span" in a trace represents one step — an LLM call, a tool call, a retrieval, etc. — with timing, inputs, outputs, and any errors
- Traces are stored in the MLflow tracking server and visible in the Databricks workspace UI
- For production monitoring, traces from live traffic are stored in **inference tables** (a Delta table in your Unity Catalog) which form the basis of ongoing monitoring metrics

Important terminology

- **Span** — one unit of work in a trace (e.g., a single LLM call, a single tool invocation)
- **Trace** — the full tree of spans for one end-to-end agent invocation
- **Inference table** — the Delta table where production traces/requests/responses are stored for monitoring

Databricks implementation

python

```python
import mlflow

# For LangChain-based agents — one line enables full tracing
mlflow.langchain.autolog()

# For manual instrumentation
with mlflow.start_span(name="retrieval_step") as span:
    span.set_inputs({"query": user_query})
    results = vector_search.query(user_query)
    span.set_outputs({"results": results})
```

Real-world example

A multi-step financial agent consists of four parallel and sequential chains. When the agent fails on step 3 of a weekend test run, the engineer enables MLflow Tracing, re-runs the agent, and immediately sees in the trace that step 3's tool call returned a schema mismatch error — root cause identified in minutes without touching the code.

Important exam points

- **"Debugging a multi-step agent chain" = MLflow Tracing** — this is the direct, supported, low-effort way to gain step-level visibility. The exam uses phrases like "difficult to debug the root cause," "find an appropriate approach to research this issue," "gain visibility into each agent's behavior and execution step" — all pointing to Tracing.
- Tracing is for **debugging and observability**, `mlflow.genai.evaluate()` is for **quality evaluation** — don't confuse them.
- In **production**, enabling AI Gateway Inference Tables captures traces automatically for monitoring without extra code.

Common mistakes/traps

- Choosing `mlflow.genai.evaluate()` when the scenario describes a debugging/root-cause problem — evaluate() measures *quality* of responses, doesn't help you find which step in a chain threw an error.
- Choosing "deconstruct the agent into independent steps to simplify debugging" — destructive and unnecessary when Tracing can be enabled non-destructively.

Example exam questions (style)

- "A Generative AI Engineer is developing an agent system using a popular agent-authoring library. The agent comprises multiple parallel and sequential chains. The agent fails at one of the steps, making it difficult to debug the root cause. They need to find an appropriate approach to research this issue. Which approach do they choose?" → Enable MLflow Tracing to gain visibility into each agent's behavior and execution step.

Topic 13: Prompt Registry & Lifecycle Management

What is it?

The MLflow 3 **Prompt Registry** is a versioned, governed store for prompt templates used across GenAI applications. Instead of hardcoding prompt strings in application code, you register them as named, versioned artifacts with a managed lifecycle.

Why is it used?

- Multiple applications share the same prompt templates — changes need to propagate consistently
- Prompts need the same lifecycle governance as models: staged development, review, controlled promotion to production, rollback if issues arise
- Version history enables auditability: "exactly what prompt was serving production last Tuesday?"

How does it work?

- Prompts are **registered** with a name and version number
- **Aliases** act as pointers to specific versions:
  - `production` → currently live version
  - `champion_backup` or similar → previous production version (for rollback)
  - `archived` → no longer maintained
  - `deprecated` → flagged as obsolete, should not be used
- Aliases are **moved** to point at new versions rather than copying prompt content

Important terminology

- **Alias** — a mutable pointer (label) that can point to any version of a registered prompt; what your application references at runtime so you can change versions without code deploys
- `mlflow.register_prompt()` — function to create a new registered prompt or a new version of an existing one
- `mlflow.load_prompt()` — function to load a specific version or alias at runtime

Databricks implementation

```python
```

```python
import mlflow

# Register a new prompt version
prompt = mlflow.genai.register_prompt(
    name="customer_support_prompt",
    template="You are a helpful support agent. Answer the following question: {{qu
    commit_message="Added de-escalation tone guidelines"
)

# In your application: always load by alias, never hardcode the version number
prompt = mlflow.load_prompt("prompts:/customer_support_prompt@production")

# Promote a validated staging version to production
mlflow.set_prompt_alias("customer_support_prompt", "production", version=5)
# Move old production to a backup alias for rollback safety
mlflow.set_prompt_alias("customer_support_prompt", "champion_backup", version=4)
# Tag the oldest unsupported version
mlflow.set_prompt_tag("customer_support_prompt", version=1, key="status", value="c
```

Lifecycle transition — the correct order (directly tested in Q8)

```
Current state:
  version 1 → alias: deprecated
  version 3 → alias: production (currently serving traffic)
  version 5 → new, validated in staging, ready to promote

Correct promotion sequence:
  1. Assign alias "production" to version 5 (new version is now live)
  2. Move alias "champion_backup" to version 3 (old production, now a rollback
target)
  3. Apply "deprecated" tag to version 1 (oldest, no longer supported)

Wrong approaches:
  ❌ Archive version 3 BEFORE version 5 is promoted (lose rollback capability)
  ❌ Create a NEW prompt name for version 5 (breaks lineage, defeats
versioning)
  ❌ Set version 3's alias to "archived" without first assigning "production"
to version 5
     (leaves production alias pointing at an about-to-be-archived version)
```

Promotion across environments (directly tested in Q11)

- The correct pattern when a prompt has been validated in staging: **version the prompt artifact in MLflow, promote through the registry's governed alias workflow**
- Wrong alternatives: sharing raw staging data with production (no versioning), exporting to a text file (no versioning, no rollback), merging via Git CI/CD pipeline (bypasses the registry's built-in purpose)

Important exam points

- **Never archive/deprecate the old production version before promoting the new one** — you lose your rollback safety net. Promote first, then update the old version's alias.
- Applications should **reference prompts by alias** (`@production`), not hardcoded version number — this is what allows zero-code-change prompt updates.
- **Registering a new prompt *name*** when you only want a new *version* of an existing prompt is wrong — it breaks lineage tracking and version history.

Common mistakes/traps

- Confusing "alias" (mutable pointer) with "version" (immutable snapshot) — versions never change once created; aliases move to point at different versions.
- Thinking "Git branch = Prompt Registry" — they're related concepts but Git manages code, the Prompt Registry manages *ML artifacts* with ML-specific lifecycle tooling (aliasing, tagging, MLflow tracking integration).

Comparison with related concepts

	Prompt Registry	**Model Registry**
What's stored	Prompt templates (text)	Model artifacts (weights, code, dependencies)
Versioning	Yes	Yes
Aliases	Yes (same concept: production, champion, archived)	Yes (same concept)
Part of MLflow?	Yes (MLflow 3 Prompt Registry)	Yes (MLflow Model Registry)

Example exam questions (style)

- "A new prompt version has been validated in staging. The previous production version must remain accessible for rollback for 30 days. Oldest versions should be flagged as no

longer supported. What is the correct sequence?" → Assign `production` alias to the new version; reassign the previous production version's alias to `champion_backup`; apply `deprecated` tag to oldest versions.

Topic 14: MLflow GenAI Evaluation — `mlflow.genai.evaluate()`

What is it?

`mlflow.genai.evaluate()` is the MLflow 3 function for systematically measuring the quality of a GenAI application (agent, RAG chain, LLM endpoint) against a labeled or unlabeled evaluation dataset.

Why is it used?

Before deploying or iterating on a GenAI application, you want objective, reproducible metrics — not just "it seemed to work well in manual testing." `mlflow.genai.evaluate()` runs your app (or a dataset of pre-collected outputs) through a set of **scorers** (built-in judges and/or custom scorers) and logs the results to an MLflow experiment for comparison.

How does it work?

```python
import mlflow

results = mlflow.genai.evaluate(
    data=eval_dataset,           # pandas DataFrame with inputs (and optionally ex
    predict_fn=my_agent,         # callable: the agent/chain to evaluate (or omit
    scorers=[                    # list of scorers to apply
        mlflow.genai.judges.correctness(),    # built-in judge (needs ground truth
        mlflow.genai.judges.groundedness(),   # built-in judge (no ground truth ne
        my_custom_scorer,                     # your own @scorer-decorated functio
    ]
)
```

Important terminology

- **Evaluation dataset** — a pandas DataFrame (or mlflow Dataset) where each row is one test case. Columns typically include: `inputs` (the question/request), `outputs` (optional, if you pre-collected responses), and `expected_response` (the known-correct answer, required for ground-truth-dependent judges)
- **Scorer** — a function that takes a completed prediction (inputs + outputs + optional ground truth) and returns a score or `Feedback` object

- **Ground truth** — a human-authored "correct" or "expected" answer used by reference-based judges to check accuracy

The evaluation dataset fields (important exact-column knowledge)

Column	Purpose	Required by
`inputs`	The question/request sent to the app	All scorers
`outputs`	The app's response (can be pre-collected or generated live)	All scorers
`expected_response`	The known-correct/expected answer	Ground-truth-dependent judges only
`retrieved_context`	Chunks the retriever returned	Context-related judges
`trace`	MLflow trace for the call	Trace-based scorers

Important exam points

- Know exactly which built-in judges need `expected_response` (ground truth) vs. which are reference-free:

Judge	Needs ground truth?	What it measures
`correctness`	✅ Yes	Is the response factually correct vs. the expected answer?
`context_sufficiency`	✅ Yes	Could the retrieved context support the expected answer?
`groundedness`	❌ No	Is the response actually supported by the retrieved context (no hallucination)?
`relevance_to_query`	❌ No	Is the response relevant to the input query?
`chunk_relevance`	❌ No	Is each retrieved chunk relevant to the query?
`guideline_adherence`	❌ No (uses stated guideline, not expected answer)	Does the response follow a stated rule/guideline?

- This table is one of the highest-return things to memorize for the exam — Q5 in your practice set tests it directly and it recurs across multiple Section 6 objectives.

Topic 15: Custom Scorers — `@mlflow.genai.scorer`

What is it?

A custom scorer is a Python function you write that evaluates a specific quality dimension not covered by built-in judges — for example, checking whether a response contains a prohibited phrase, follows a company-specific tone policy, or includes a required legal disclaimer.

Why is it used?

Built-in judges cover general dimensions (correctness, groundedness, relevance). But every real-world application has unique quality criteria — a custom scorer lets you encode those criteria as code that runs automatically inside the same `mlflow.genai.evaluate()` workflow.

How does it work? (MLflow 3 — exact API, directly tested)

Method 1: `@mlflow.genai.scorer` decorator (PRIMARY — recommended for most cases)

python

```python
from mlflow.genai.scorers import scorer, Feedback

@scorer
def tone_policy_scorer(inputs, outputs) -> bool:
    """Returns True if the response contains no prohibited phrases."""
    prohibited = ["I don't know", "I cannot help", "As an AI"]
    response_text = outputs.get("content", "")
    return not any(phrase in response_text for phrase in prohibited)

# Can also return a Feedback object for more detail:
@scorer
def tone_policy_scorer_v2(inputs, outputs) -> Feedback:
    prohibited = ["I don't know", "I cannot help"]
    response_text = outputs.get("content", "")
    passed = not any(phrase in response_text for phrase in prohibited)
    return Feedback(
        score=passed,
        rationale=f"Response {'passed' if passed else 'failed'} tone policy check"
    )

# Use it:
mlflow.genai.evaluate(data=eval_df, predict_fn=my_agent, scorers=[tone_policy_scor
```

Method 2: `Scorer` class (for offline evaluation only)

```python
from mlflow.genai.scorers import Scorer, Feedback

class TonePolicyScorer(Scorer):
    name = "tone_policy"

    def score(self, inputs, outputs, **kwargs) -> Feedback:
        prohibited = ["I don't know"]
        passed = not any(p in outputs.get("content", "") for p in prohibited)
        return Feedback(score=passed, rationale="Tone check")

# Use it:
mlflow.genai.evaluate(data=eval_df, predict_fn=my_agent, scorers=[TonePolicyScorer
```

Why Method 1 (`@scorer`) is the primary/preferred approach

- `@scorer`-decorated functions can be used in **both** offline evaluation AND **production monitoring** (registered as production scorers)
- `Scorer`-class-based scorers are for **offline evaluation only** — they cannot be registered for production monitoring
- This distinction is directly tested — your Q13's correct answer (A) specifically uses the `@scorer` decorator with `mlflow.genai.evaluate()` via the `scorers` parameter

What NOT to use (legacy MLflow 2.x patterns — wrong answers in your set)

```python
# ❌ WRONG for MLflow 3 — this is a legacy mlflow.evaluate() pattern (mlflow 2.x)
mlflow.evaluate(model, extra_metrics=[mlflow.metrics.genai.make_genai_metric(...)]

# ❌ WRONG — mlflow.genai.make_genai_metric doesn't exist in MLflow 3's GenAI API
mlflow.metrics.genai.make_genai_metric(...)

# ❌ WRONG — mlflow.entities.AssessmentSource used this way is not the documented
```

Important exam points

- The parameter name for passing scorers to `mlflow.genai.evaluate()` is **`scorers=`** (not `extra_metrics=`, not `custom_metrics=`)
- Return type from a scorer function: a `bool`, a numeric value, or a `Feedback` object (which adds a rationale string)
- `@scorer` decorator = use anywhere (offline eval + production monitoring); `Scorer` class = offline eval only

Common mistakes/traps

- Picking answers that reference `make_genai_metric` or `extra_metrics` — these are MLflow 2.x patterns; wrong on an MLflow 3 exam.
- Thinking the `Scorer` class is the "more complete/correct" version for all scenarios — it's more limited in deployment scope than `@scorer`.

Example exam questions (style)

- "A Generative AI Engineer wants to define a custom scorer using MLflow 3 that programmatically checks if a response contains prohibited phrases and returns a binary score. The scorer must be registered and reusable across multiple evaluation runs. Which approach is correct?" → Define a Python function decorated with `@mlflow.genai.scorer` that accepts inputs and outputs and returns a bool or Feedback, then pass it to `mlflow.genai.evaluate()` via the `scorers` parameter.

Topic 16: Evaluation vs. Monitoring — the Lifecycle Distinction

What is it?

Two separate phases of the GenAI application lifecycle, each with different data sources, tools, timing, and goals:

Dimension	Evaluation Phase	Monitoring Phase
Timing	Pre-production (before deployment)	Post-deployment (in production)
Data source	Curated evaluation dataset (human-labeled)	Live inference tables / production traces
Primary tool	`mlflow.genai.evaluate()` with scorers/judges	Databricks Lakehouse Monitoring + inference tables
Goal	Assess readiness and quality BEFORE going live	Detect quality degradation and operational issues AFTER going live
Metrics computed from	Expected responses in eval dataset	Live traffic sampled from inference tables

Why does this distinction matter for the exam?

Your Q23 tests it directly, and it's a Section 3 official exam objective ("Compare the evaluation and monitoring phases of the GenAI application lifecycle within Databricks"). The wrong answers mix up the data sources (e.g., claim evaluation uses live traffic, or monitoring uses a curated eval dataset).

How does monitoring work in Databricks?

- Deploying an agent through Databricks Model Serving with the Mosaic AI Agent Framework automatically enables **inference table** capture of request/response/trace data
- You then configure **Lakehouse Monitoring** on that inference table to compute quality and operational metrics periodically
- The monitoring phase uses **inference logs** and **traces** from live traffic as its data source — not a pre-built evaluation set

Important exam points

- "**Evaluation dataset**" (with `expected_response`) = belongs to the **evaluation phase** (pre-production)
- "**Inference tables**" / "**live API calls**" / "**production traces**" = belong to the **monitoring phase** (post-deployment)
- The MLflow tool used for the **evaluation phase** is `mlflow.genai.evaluate()` (or Mosaic AI Agent Evaluation in the UI)

- The Databricks features used for the **monitoring phase** are Inference Tables + Lakehouse Monitoring

Common mistakes/traps

- Answer option claiming "the evaluation phase uses all inference history to assess agent performance and readiness for production" — this is wrong; evaluation uses a *curated dataset*, not live inference history.
- Answer option claiming "the monitoring phase runs metrics on inference table records that only contain new records" — also wrong; monitoring can process historical inference data too.

Topic 17: LLM Evaluation Metrics

The key metric pairs the exam tests

BLEU vs. ROUGE

	BLEU	**ROUGE**
Full name	Bilingual Evaluation Understudy	Recall-Oriented Understudy for Gisting Evaluation
Designed for	**Machine translation**	**Summarization**
What it measures	Precision of n-gram overlaps (generated vs. reference)	Recall of n-gram overlaps (generated vs. reference)
Requires ground truth?	Yes (reference translations)	Yes (reference summaries)

Memory trick: BLEU → translation (B = Bilingual), ROUGE → summarization (R = Recall of summaries)

Retrieval metrics

	Recall	NDCG
What it measures	Fraction of truly relevant items retrieved	Quality of ranking — rewards relevant items being ranked higher
Asks	"Did we find it at all?"	"Did we rank the best ones at the top?"
Use case	Did the retriever surface every relevant chunk?	Is the ordering of retrieved results good?

MMLU — the benchmark trap

- **MMLU (Massive Multitask Language Understanding)** is a popular academic benchmark used to compare foundation models during the model *selection* phase
- It is **NOT** a production monitoring metric — you don't compute MMLU against your live traffic
- "Which is NOT a metric for monitoring a production LLM application?" → MMLU is the correct (trap-detecting) answer

Production operational metrics (what you DO monitor)

- Request volume (how many queries per hour/day)
- Latency (how long to generate a response)
- Error rate (what fraction of requests fail)
- Token usage / cost
- Quality metrics from scheduled evaluation runs (correctness, groundedness, etc. — computed periodically against sampled live traffic, not continuous)

Common mistakes/traps

- Applying BLEU/ROUGE to retrieval quality evaluation — these are text-generation metrics, not embedding-similarity metrics
- Confusing MMLU (pre-deployment benchmark) with production monitoring metrics
- Assuming operational metrics are "quality metrics" — they measure system health, not response quality

Example exam questions (style)

- "An LLM system for translation is being benchmarked. Which metric?" → BLEU
- "Summarization quality evaluation against reference summaries?" → ROUGE
- "Retrieval ranking quality where position matters?" → NDCG

- "Which metric is NOT appropriate for production monitoring of a customer service LLM?" → MMLU (it's a benchmark, not a live-traffic metric)

Topic 18: SME-Based Evaluation and Rubric Calibration

What is it?

Human subject matter experts (SMEs) review agent responses and provide quality ratings (e.g., "Correct/Incorrect," 1-5 scale, "Helpful/Not Helpful"). These ratings become the **ground truth** that calibrates automated evaluators (LLM-as-judge scorers) and drives model improvement.

Why does this matter?

SMEs frequently disagree with each other — different people have different implicit quality standards. If you use inconsistent human labels as ground truth for training or evaluation, you get an unreliable benchmark. The fix isn't to abandon human judgment — it's to **align SMEs on explicit rubrics first**, then use their consistent, calibrated judgments.

How does it work?

1. Identify a representative sample of agent responses where SMEs disagree
2. Bring SMEs together to review those responses and discuss *why* they rated them differently
3. Document the agreed-upon criteria as explicit **rubrics** ("A response is 'Correct' if it names the specific policy section and doesn't add unverified claims")
4. Re-label the sample using the agreed rubrics — use the high-agreement subset as the benchmark going forward
5. Implement `mlflow.genai.evaluate()` with scorers calibrated against this rubric-aligned ground truth

Important exam points (directly tested in Q54)

The hierarchy of ground truth:

```
Human SME judgment (properly calibrated via rubrics) > LLM-as-a-judge scores
```

- LLM-as-a-judge is a **scaling tool** to apply consistent rubrics to large volumes of data — it should be *calibrated against* well-aligned human judgment, not used to override or replace it
- Wrong: "Use an LLM-as-judge to rescore everything and treat those scores as the ground truth" — this inverts the appropriate hierarchy

- Right: "Develop clear rubrics with SMEs to align judgment standards, then implement `mlflow.genai.evaluate()` using those standards"

Common mistakes/traps

- Averaging inconsistent SME scores — masks disagreement statistically, produces a grey zone that doesn't reflect actual quality standards
- Filtering the eval set to only the responses where SMEs already agreed — wastes data and doesn't fix future disagreement
- Using an LLM-as-judge to resolve SME disagreements — an LLM has its own biases and cannot be the arbiter of human domain expertise

Example exam questions (style)

- "Three SMEs rating agent responses frequently disagree. The engineer needs a robust evaluation framework for iterative improvement. What should they implement?" → Develop clear rubrics with SMEs to align judgment standards; use that calibrated, high-agreement subset as the benchmark for `mlflow.genai.evaluate()`.

Quick self-check before moving on

1. What is a trace and what is a span? When do you use MLflow Tracing vs. `mlflow.genai.evaluate()`?
2. Name three built-in judges that do NOT need ground truth, and two that do.
3. Write the skeleton of a `@scorer`-decorated custom scorer function.
4. What parameter name does `mlflow.genai.evaluate()` use for scorers?
5. What does `production` alias point to in the Prompt Registry, and why shouldn't you archive the old production version before the new one is promoted?
6. BLEU is for ___, ROUGE is for ___, NDCG is for ___, MMLU is a ___ not a ___.
7. Why is "use an LLM-as-judge to resolve SME disagreements" wrong?

Next: Study Material Part 4 — Model Serving, Foundation Model APIs, Governance & Responsible AI (Sections 4, 5 & 6 objectives for deployment, cost, authentication, guardrails, and Unity Catalog).